

## Punto 5.1 — Concurrencia optimista: concepto y necesidad

### Contexto del proyecto

Este punto continúa directamente M04/PROYECTO/4.12. La carpeta M05/PROYECTO/5.1 contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo no cambia todavía; se conserva la historia heredada del Módulo 4.

### Objetivo práctico

Reproducir una actualización perdida real sobre la misma propiedad y contrastarla con cambios concurrentes sobre propiedades distintas.

### Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.1
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

### Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/ConcurrenciaOptimistaM5Repositorio.cs`
- `src/AceriaData.Application/ConcurrenciaOptimistaM5UseCase.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

### Paso 3: Implementar y estudiar `ConcurrenciaOptimistaM5Repositorio.cs`

```
public ConcurrenciaMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()
{
    RestaurarEstadoInicial();

    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();

    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    SqlCommandCounterInterceptor.Instance.Reset();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

    var clienteInicial = ordenA.Cliente;

    ordenA.Cliente = "Cliente actualizado por A";
    contextA.SaveChanges();

    ordenB.Cliente = "Cliente actualizado por B";
    contextB.SaveChanges();

    using var scopeVerificacion = _scopeFactory.CreateScope();
    var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var clienteFinal = contextVerificacion.OrdenesFabricacion
        .AsNoTracking()
```

```

        .Where(o => o.NumeroOrden == NumeroOrden)
        .Select(o => o.Cliente)
        .Single();

    return new ConcurrencyMismaPropiedadDto(
        clienteInicial,
        ordenA.Cliente,
        ordenB.Cliente,
        clienteFinal,
        clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

### Explicación línea a línea — ConcurrencyOptimistaM5Repositorio.cs

Línea 1: `public ConcurrencyMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarEstadoInicial();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 5: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 8: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 9: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 11: `SqlCommandCounterInterceptor.Instance.Reset();` → Reinicia el contador para que solo se midan los comandos de este escenario.

Línea 13: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 14: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 16: `var clienteInicial = ordenA.Cliente;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 18: `ordenA.Cliente = "Cliente actualizado por A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 21: `ordenB.Cliente = "Cliente actualizado por B";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 22: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 24: `using var scopeVerificacion = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 25: `var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 26: `var clienteFinal = contextVerificacion.OrdenesFabricacion` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 27: `.AsNoTracking()` → Ejecuta una lectura sin dejar entidades en el `ChangeTracker`.

Línea 28: `.Where(o => o.NumeroOrden == NumeroOrden)` → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.

Línea 29: `.Select(o => o.Cliente)` → Proyecta únicamente la forma de datos necesaria para el resultado.

Línea 30: `.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 32: `return new ConcurrenciaMismaPropiedadDto()` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 33: `clienteInicial,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 34: `ordenA.Cliente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 35: `ordenB.Cliente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 36: `clienteFinal,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 37: `clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 38: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 39: `}` → Delimita el bloque sintáctico correspondiente.

## Paso 4: Implementar y estudiar `ConcurrenciaOptimistaM5Repositorio.cs`

```
public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()
{
    RestaurarEstadoInicial();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

    ordenA.Cliente = "Cliente actualizado por A";
    contextA.SaveChanges();
    ordenB.Estado = "EnProceso";
    contextB.SaveChanges();

    using var scopeVerificacion = _scopeFactory.CreateScope();
    var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var resultado = contextVerificacion.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.NumeroOrden == NumeroOrden)
        .Select(o => new { o.Cliente, o.Estado })
        .Single();

    return new ConcurrenciaPropiedadesDistintasDto(
        ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,
        resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

**Explicación línea a línea — ConcurrenciaOptimistaM5Repositorio.cs**

Línea 1: `public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarEstadoInicial();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 9: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `ordenA.Cliente = "Cliente actualizado por A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 13: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 14: `ordenB.Estado = "EnProceso";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 17: `using var scopeVerificacion = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 18: `var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 19: `var resultado = contextVerificacion.OrdenesFabricacion` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 20: `.AsNoTracking()` → Ejecuta una lectura sin dejar entidades en el `ChangeTracker`.

Línea 21: `.Where(o => o.NumeroOrden == NumeroOrden)` → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.

Línea 22: `.Select(o => new { o.Cliente, o.Estado })` → Proyecta únicamente la forma de datos necesaria para el resultado.

Línea 23: `.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 25: `return new ConcurrenciaPropiedadesDistintasDto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 26: `ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 28: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 29: `}` → Delimita el bloque sintáctico correspondiente.

## Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.1 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

## Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Dos DbContext independientes cargan la misma orden antes de guardar.
- La misma propiedad permite observar una actualización perdida sin token.
- Propiedades distintas se conservan cuando EF Core genera UPDATE de las propiedades modificadas.
- La verificación final usa un tercer contexto AsNoTracking.
- El SQL mostrado procede del interceptor real.

## Reto resuelto y ampliación

**Reto:** Añade una tercera escritura concurrente sobre Cliente y comprueba, mediante el SQL capturado, qué valor queda finalmente sin token.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

## Errores comunes

- Usar el mismo DbContext para ambos actores y creer que se reproduce concurrencia.
- Modificar propiedades distintas y afirmar que necesariamente existe actualización perdida.
- Leer el resultado final desde un contexto con estado antiguo en el ChangeTracker.
- Inventar un SQL esperado en vez de observar el comando ejecutado.

## Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.1 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

## Resultado esperado

Al finalizar el punto 5.1, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.1 0K

## **Conexión con el siguiente punto**

El siguiente estado es 5.2 y parte directamente de este proyecto completo.

---